

FINAL YEAR PROJECT REPORT

Smart Fire Detection & Automatic Suppression System for Electrical Distribution Boards

A low-cost, AI-assisted IoT solution that senses, predicts, reports, and automatically suppresses fire risk inside electrical distribution boards — before it becomes a fire.

PROJECT TEAM

230491J Piriyaatharsan M.

230496E Prabhath U.L.K.

230504F Pussalla P.G.D.C.

1 Project Scope

Electrical distribution boards (DBs) are one of the most common origins of building fires, yet the protection devices installed in almost every DB — MCBs, RCCBs, and fuses — are not designed to detect fire risk itself. This project defines a system that watches the conditions that lead to fire, not just the electrical fault that eventually follows.

Why Distribution Boards Are a Fire Hazard

COMMON ROOT CAUSES

- Loose electrical connections
- Overloaded circuits
- Damaged insulation
- Arc faults & short circuits
- Component aging, dust build-up, high ambient heat

HOW A DB FIRE DEVELOPS

Electrical fault → Heat generation
→ Smoke → Flame → Complete
DB fire

Most DB fires start internally, well before anyone in the building notices.

Why Existing Protection Isn't Enough

Protection Device	Protects Against	Cannot Detect
MCB	Overcurrent	Slow overheating
RCCB	Earth leakage	Smoke
Fuse	Excess current	Internal fire
Our System	Heat + Smoke + Current + AI	Early fire prevention

MCBs only trip once current exceeds a rated threshold — but loose terminals generate heat at normal current, arc faults often stay under that rating, and smouldering insulation produces smoke long before current rises. Even after an MCB trips, hot residual material can keep a fire going. In short: **electrical protection devices protect circuits, not fires.**

Where Existing Suppression Falls Short

DIN-RAIL AEROSOL EXTINGUISHERS (MECHANICAL SYSTEM)

Advantages	Limitations
------------	-------------

- | | |
|------------|--|
| • Fast | • Activates only after fire is already significant |
| • Reliable | • No monitoring, no remote alerts, no prediction |

Project Objectives

- Detect fire risk before flame develops
- Reduce potential damage and downtime
- Continuously monitor the distribution board's condition
- Notify users remotely, in real time
- Automatically suppress fire once confirmed
- Provide AI-assisted risk prediction on top of raw sensor thresholds

2 What We Did — Implementation

The system was built in three connected layers: sensing & actuation hardware, LabVIEW control logic running on a DAQ device, and a cloud-connected web/AI/notification layer.

2.1 Sensors & Actuators

SENSORS USED

- Temperature Sensor — LM35
- Flame Sensor — KY-026
- Smoke Sensor — custom-built using an LED + LDR
- Current Sensor — SCT-013-100

ACTUATORS USED

- Cooling Fan (speed-controlled)
- Buzzer alarm
- Suppression motor / relay (suppression prototype)

2.2 Custom Smoke Sensor Design

Since no ready-made smoke sensor was used, the smoke detector was engineered from scratch through several iterations:

1. **IR LED + LDR** — worked in principle, but IR light is invisible, making alignment impossible to verify visually.

2. **Visible LED & LDR facing directly** — alignment could now be confirmed, but placing them opposite each other saturated the LDR, so smoke barely changed the output voltage.
3. **Visible LED & LDR at 90°** — light no longer hits the LDR directly; smoke particles scatter light sideways into the LDR instead, giving a clean, sensitive voltage response.

The design was then fine-tuned: a small amount of ambient light was allowed into the chamber (to stop the LDR sitting at its dark-saturation extreme), and LED brightness was calibrated so even light smoke registered clearly without re-saturating the sensor.

2.3 Current Sensing & Calibration

The SCT-013-100 current sensor's output (voltage across a burden resistor) was calibrated against known current values in LabVIEW using DAQmx analog input logging, producing a calibration curve used to convert sensed voltage back into amperes — for example, 1.0 A corresponds to roughly 0.447 V, and 1.8 A to roughly 0.812 V across the burden resistor.

2.4 LabVIEW Control Logic

FAN COOLING CONTROL

Temperature sensors → LabVIEW logic → DAQ PWM output → L298N motor driver
→ Fan

LabVIEW compares inside vs. outside temperature and selects a duty cycle (0% / 30% / 60% / 100%). Since the DAQ's digital output cannot supply the fan's current directly, an L298N motor driver sits in between: the DAQ provides the control signal (PWM into ENA), and the L298N supplies the actual driving power to the fan.

AUTOMATIC FIRE SUPPRESSION

DAQ digital output → MOSFET switch → Relay coil → Suppression device

A MOSFET acts as a low-side electronic switch, since the DAQ cannot drive a relay coil's current directly. When LabVIEW sets suppression TRUE, the DAQ output goes HIGH, switching the MOSFET on, energizing the relay coil, closing its normally-open contact, and powering the suppression device. A flyback diode across the coil protects the MOSFET from the voltage spike generated when the coil switches off.

BUZZER ALARM LOGIC

A single Boolean (Buzzer TRUE/FALSE) drives a Case Structure in LabVIEW. The TRUE case sends a HIGH signal to the DAQ digital output; the FALSE case sends LOW. Since the buzzer needs more current than the DAQ can supply, the digital output is used purely as a control signal, driving the buzzer through a transistor/relay stage.

2.5 From Bluetooth to a Cloud API

The original communication plan was:

LabVIEW → VISA Write → Arduino Uno → HC-05 Bluetooth Module

VISA Read performed well, but **VISA Write introduced a noticeable delay**, which prevented smooth real-time data transfer. The team redesigned the communication layer around LabVIEW's built-in HTTP Client instead:

LabVIEW → HTTP PUT / GET (JSON) → API Endpoint → Supabase (PostgreSQL) → Website & Telegram Bot

In effect, LabVIEW "fills out an online form" every few seconds via an HTTP PUT request, while the website and Telegram bot simply "check the form" (GET) whenever they need the latest values — no pairing, no cables, just a standard internet connection.

EXAMPLE SENSOR PAYLOAD (JSON)

```
{
  "temperatureInside": 26,
  "temperatureOutside": 24,
  "smoke": 110,
  "flame": 0,
  "current": 1.0,
  "fanSpeed": 40,
  "buzzer": false,
  "fireSuppression": false
}
```

2.6 Web Application, Database & Telegram Bot

Technology	Used For
TypeScript + React (TSX)	Web application frontend

Node.js + TypeScript	Telegram bot
Supabase	Backend service
PostgreSQL	Actual database
SQL	Database migrations
CSS	Styling

The live web dashboard displays sensor readings, fan/buzzer/suppression states, overall system status, and actuator error flags. Critically, the website independently recalculates what each actuator *should* be doing from its own logic and compares that against what LabVIEW actually reports — a mismatch (e.g., logic says the fan should be ON, but the reported state is OFF) is flagged as an actuator error, helping catch a faulty or disconnected component early.

The Telegram bot lets a user check all sensor readings and system/actuator status on demand, and — more importantly — pushes an alert automatically whenever an important condition occurs, without the user needing to ask.

2.7 AI-Based Fire Risk Prediction

The risk model watches two correlated trends: the **current sensor reading** and the **temperature difference between inside and outside the DB**. If current is rising while the inside temperature climbs much faster than the outside temperature, the system raises a fire risk warning — often before smoke or flame sensors cross their own danger thresholds.

Example: outside temperature holds steady at 24°C, while inside temperature rises from 26°C to 35°C in a few minutes and current climbs from 1.0 A to 2.5 A at the same time — the AI flags rising fire risk from the combined trend, ahead of a smoke or flame alarm.

ONE-LINE SUMMARY

LabVIEW senses and controls the hardware, the website stores and monitors the data, the AI layer predicts fire risk from sensor trends, and the Telegram bot delivers instant alerts to the user.

3 Benefits of the System

Because the system reasons about heat, smoke, and current trends together — instead of reacting to a single overcurrent threshold — it offers protection that conventional DB safety devices simply cannot.

Early Fire Prevention

Detects the conditions that lead to fire — heat build-up, smoke, and abnormal current trends — well before flame develops, unlike MCBs/RCCBs which only react to overcurrent or earth leakage.

Reduced Damage & Downtime

Catching a fault at the smouldering/overheating stage avoids the far greater cost, downtime, and safety risk of a full DB fire.

Remote, Real-Time Monitoring

The live web dashboard and Telegram bot mean the DB's condition can be checked from anywhere, with no need to be physically present at the panel.

Automatic, Instant Response

Cooling fan, buzzer, and fire suppression actuation are all triggered automatically by LabVIEW logic — no manual intervention needed in the critical first moments.

Actuator Health Verification

The website's independent expected-vs-reported state check catches a faulty or disconnected fan, buzzer, or suppression actuator before it becomes a real gap in protection.

AI-Assisted Early Warning

Combining current and temperature-differential trends gives a predictive warning ahead of traditional threshold-based smoke/flame alarms.

Low-Cost, Accessible Design

Built from low-cost components (LM35, KY-026, an LDR-based smoke sensor, SCT-013-100) and a DAQ + LabVIEW platform, rather than expensive commercial fire-monitoring hardware.

Data Logging for Analysis

Sensor data logged through the DAQ/TDMS pipeline and the cloud database supports later trend analysis, calibration, and further AI model improvement.

4 Issues Faced & How We Overcame Them

Every subsystem went through a real trial-and-error process. The build log below captures the main issues encountered, why they happened, and the fix applied.

SMOKE SENSOR IR alignment could not be verified

Issue: An IR LED + LDR pair was used first, but IR light is invisible, so beam alignment onto the LDR could not be checked by eye.

Fix: Replaced the IR LED with a visible LED for testing, allowing the beam direction and alignment to be confirmed visually.

SMOKE SENSOR LDR saturation with direct-facing LED

Issue: With the LED and LDR placed directly opposite each other, smoke produced only a very small voltage change.

Why: Direct light already hit the LDR at full intensity, saturating it, so extra light scattered by smoke couldn't add a meaningfully larger signal.

Fix: Repositioned the LED and LDR at a 90° angle so light only reaches the LDR when scattered by smoke particles, giving a much cleaner, more sensitive response.

SMOKE SENSOR Darkness saturation & weak smoke signal

Issue: In full darkness the LDR sat at the opposite extreme, and small smoke amounts sometimes produced no clear voltage change.

Fix: Allowed a small amount of ambient light into the chamber to keep the LDR in its active operating range, and increased LED brightness just enough that light smoke scattered a detectable signal without re-saturating the sensor.

COMMUNICATION Bluetooth (VISA Write) delay

Issue: The original LabVIEW → Arduino Uno → HC-05 Bluetooth link, using VISA Write, introduced a noticeable delay that broke real-time data delivery.

Why: VISA Read was fast, but VISA Write over the serial/Bluetooth link was slow, making the data feel laggy on the dashboard.

Fix: Replaced the Bluetooth link entirely with LabVIEW's HTTP Client, sending JSON sensor/actuator data via HTTP PUT to a Supabase-backed API — removing pairing and serial-delay issues altogether.

FAN CONTROL DAQ output cannot drive the fan directly

Issue: The DAQ digital/PWM output only provides a small control signal — not enough current to run the fan motor.

Fix: Added an L298N motor driver between the DAQ and the fan: the DAQ supplies the PWM control signal into ENA, while the L298N supplies the actual driving power from an external supply. A shared ground between the DAQ and L298N was required for the PWM signal to be interpreted correctly.

Extra fix: The L298N's ENA jumper had to be removed; leaving it in place locks ENA HIGH permanently, so the fan only ever runs at full speed and PWM speed control has no effect.

FIRE SUPPRESSION DAQ output cannot drive a relay coil

Issue: Energizing a relay coil needs more current than the DAQ's digital output can supply directly.

Fix: Used an N-channel MOSFET as a low-side electronic switch: the DAQ drives the MOSFET's gate, and the MOSFET switches the relay coil's higher current from an external supply.

Extra fix: Added a flyback diode across the relay coil, since the coil is an inductive load that generates a reverse voltage spike when switched off — protecting the MOSFET and DAQ from damage.

ACTUATOR VERIFICATION No way to detect a faulty actuator

Issue: If a fan, buzzer, or suppression actuator failed silently, LabVIEW's report of its state could be wrong with nothing to flag the mismatch.

Fix: Added independent expected-state logic on the website: it recalculates what each actuator should be doing from the live sensor data, compares that with the state LabVIEW reports, and raises an actuator error whenever the two disagree.

Build-log keywords: LDR saturation · light scattering · 90° arrangement · active operating range · PWM duty cycle · common ground · L298N ENA jumper · MOSFET low-side switch · flyback diode · relay NO contact · VISA Write delay · HTTP Client · actuator error flagging